Programming Principles I

Week 1

Getting Started with C

> _

IDE · Command Line · Variables · Standard I/O · Math Functions

```
printf("Hello, C!");
```

Today’s roadmap

What students should be able to do after Week 1

1. Set up the tools
2. Write a tiny C program
3. Compile from the terminal
4. Use variables and expressions
5. Read input and print output
6. Use basic math functions

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double r;
    printf("Radius: ");
    scanf("%lf", &r);
    printf("Area = %.2f\n", 3.14159 * r * r);
    return 0;
}
```

Core idea: programming is not memorizing syntax. It is translating a problem into precise steps that a machine can execute.

C

What is a program?

A program is an exact recipe written for a very literal machine

Human instruction

“Calculate the area of a circle.”

Algorithm

1. Read radius
2. Compute πr^2
3. Print result

C code

```c
scanf("%lf", &r);
area = PI * r * r;
printf("%.2f", area);
```

Bad instruction: “make it work”. Good instruction: every input, operation, and output is specified.

Question: What information is missing if I only say “calculate salary”?

From source code to running program

The computer cannot run C source code directly




```
gcc hello.c -o hello
./hello
```

Important: an error message is not a failure. It is feedback from the compiler.

Class habit: compile often, read the first error carefully, fix one thing at a time.

Week 1 setup checklist

Everything must be ready before solving classroom tasks

Install

✓ **Visual Studio Code**	Editor
✓ **GCC compiler**	Builds C programs
✓ **C/C++ extension**	Syntax + diagnostics
✓ **Terminal access**	Run commands
✓ **Git optional**	Version control later

```c
// quick test file: test.c
#include <stdio.h>

int main(void) {
    printf("C setup works!\n");
    return 0;
}
```

```
gcc test.c -o test
./test
```

Checkpoint question: What does gcc produce after a successful compile?

Command line basics

A few commands are enough for this course

```
pwd
```

Where am I?

```
mkdir week1
```

Create folder

```
ls / dir
```

What files are here?

```
gcc file.c -o app
```

Compile

```
cd folder
```

Move into a folder

```
./app
```

Run

Mini-task: create a folder named c_week1, create hello.c inside it, compile it, and run it.

Common mistake Compiling from the wrong folder. Use pwd and ls/dir first.

First C program: anatomy

Every symbol has a role

```c
#include <stdio.h>

int main(void) {
    printf("Hello, ADA!\n");
    return 0;
}
```

#include <stdio.h> gives access to printf and scanf.

main is where execution starts.

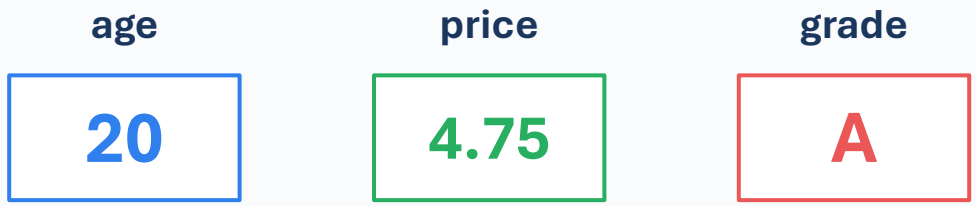
Curly braces { } define the block of code inside main.

return 0 means the program finished normally.

Quick question: What happens if we forget the semicolon after printf?

Variables: named boxes in memory

A variable stores a value so the program can use it later




```
int age = 20;
double price = 4.75;
char grade = 'A';
```

Declaration: creates the variable and gives it a type.

Assignment: stores or replaces the value.

```
int count;       // declaration
count = 5;        // assignment
count = count+1; // update
```

The assignment trap

In C, = does not mean “is equal to”

```c
int x = 5;
x = x + 1;
printf("%d\n", x);
```

Step-by-step execution

1. Create x and store 5
2. Evaluate right side: x + 1 → 6
3. Store 6 back into x

Predict: What is printed?

Now predict: int y = 3; y = y * 2 + 1;

Rule: the right side is calculated first, then the result replaces the left-side variable.

Standard output: printf

Printing is how the program communicates results

```c
int age = 20;
double gpa = 3.75;
char letter = 'A';

printf("Age: %d\n", age);
printf("GPA: %.2f\n", gpa);
printf("Grade: %c\n", letter);
```

Format specifiers

`%d`	int
`%f`	float/double
`%.2f`	2 decimals
`%c`	character
`%s`	string later

Question: Which line prints exactly two digits after the decimal point?

Style habit: Always print labels, not just raw numbers.

Standard input: scanf

Reading input means copying user data into variables

```c
int age;
double height;

printf("Enter age: ");
scanf("%d", &age);

printf("Enter height: ");
scanf("%lf", &height);
```

scanf needs the address of the variable. That is why we write &age.

For double input, use %lf in scanf. For printf, %.2f is fine.

Common beginner bug: forgetting & before the variable name.

```c
scanf("%d", age);  // wrong
scanf("%d", &age); // correct
```

Predict: What two values are stored if the user enters: 19 1.82 ?

Expressions and operators

C calculates expressions using operator rules

```c
int a = 7;
int b = 3;

printf("%d\n", a + b);
printf("%d\n", a / b);
printf("%d\n", a % b);
printf("%d\n", 2 + 3 * 4);
```

What prints?

a + b → ?

a / b → ?

a % b → ?

2 + 3 * 4 → ?

Integer division cuts off the decimal part: 7 / 3 is 2, not 2.333.

Modulo gives the remainder: 7 % 3 is 1.

Math functions

Use <math.h> for common mathematical operations

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double x = 16.0;
    printf("sqrt = %.2f\n", sqrt(x));
    printf("power = %.2f\n", pow(2, 5));
    printf("round = %.0f\n", round(3.6));
    return 0;
}
```

Useful functions

`sqrt(x)`	square root
`pow(a,b)`	a raised to b
`round(x)`	nearest integer
`ceil(x)`	round up
`floor(x)`	round down

Compile note: on some systems you may need gcc file.c -lm

Question: Which function should calculate the diagonal of a rectangle?

Worked example: circle calculator

Small program, complete problem-solving process

Problem

Read the radius of a circle and print its circumference and area.

Plan

1. Input r
2. area = π × r × r
3. circumference = 2 × π × r
4. Print both rounded to 2 decimals

Extension: Also print the diameter.

```c
#include <stdio.h>

int main(void) {
    double r, area, c;
    const double PI = 3.14159;

    printf("Radius: ");
    scanf("%lf", &r);

    area = PI * r * r;
    c = 2 * PI * r;

    printf("Area: %.2f\n", area);
    printf("Circumference: %.2f\n", c);
    return 0;
}
```

Debugging mindset

Bugs are normal; random changes are not debugging

```c
#include <stdio.h>

int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", a, b);
    printf("Sum = %d\n", a + b)
    return 0;
}
```

Find two errors

Syntax error: missing semicolon after printf.

Input bug: scanf needs &a and &b.

Debugging rule: read the first compiler error first. Later errors may be side effects.

Exercise: correct the code and test it with 4 and 9.

C

Quick checks

Use these to verify understanding before practice

Q1 What is the difference between source code and an executable?

Q2 What does #include <stdio.h> give us?

Q3 Why does scanf usually use & before a variable?

Q4 What is printed by: int x = 4; x = x + 3; printf("%d", x);

Q5 Why is 7 / 2 equal to 3 in C when both values are integers?

Q6 Write one line that prints a double with 2 decimal places.

Practice lab: three small programs

Students should complete these during or after class

1. Temperature converter

Input Celsius. Print Fahrenheit using F = C × 9 / 5 + 32.

2. Bill splitter

Input total bill and number of people. Print how much each person pays.

3. Distance formula

Input x1, y1, x2, y2. Print sqrt((x2-x1)^2 + (y2-y1)^2).

```
Checklist before submission:
[ ] Program compiles
[ ] Output has labels
[ ] Tested with at least 2 inputs
[ ] Code is readable
```

Challenge: handle money with exactly 2 digits after the decimal point.

Exit ticket: explain in one sentence why scanf needs &.

Wrap-up

What students should now be able to do

- ✓ Create and run a C program
- ✓ Compile from the terminal
- ✓ Use variables and assignments
- ✓ Print formatted output
- ✓ Read basic input with scanf
- ✓ Use arithmetic and math functions

Before next class

Finish the three practice programs. Bring your laptop ready to code. Review data types and conditionals.

Next week: data types and conditionals